Relatório de Atividade Prática: Propriedades ACID

Gustavo Borges Sousa

12 de Abril de 2026

1 Exercício Prático: Implementação de Transação

O objetivo deste exercício foi a implementação de uma transação em Python para garantir a integridade dos dados em um processo de matrícula acadêmica.

1.1 Código Python Desenvolvido

Abaixo apresento o script que desenvolvi para realizar a matrícula aplicando os conceitos de *start transaction*, *commit* e *rollback*.

```python
import mysql.connector

try:
    conexao = mysql.connector.connect(
        host="127.0.0.1", user="root", password="root123", database="aula1"
    )
    cursor = conexao.cursor()
    conexao.start_transaction()

    id_aluno_teste = 1
    id_curso_teste = 1

    # Verificando Consist ncia
    cursor.execute("SELECT COUNT(*) FROM ALUNOS WHERE id_aluno = %s", (id_aluno_teste,))
    if cursor.fetchone()[0] == 0:
        raise Exception("ERRO: Aluno n o cadastrado.")

    # Opera  o (Atomicidade)
    sql = "INSERT INTO MATRICULAS (id_aluno, id_curso, semestre, disciplina) VALUES (%s, %s, %s, %s)"
    cursor.execute(sql, (id_aluno_teste, id_curso_teste, "2026.1", "Sistemas Operacionais"))

    conexao.commit()
    print("    Sucesso!")
except Exception as e:
    if 'conexao' in locals(): conexao.rollback()
    print(f"    Falha: {e}")
```

Listing 1: matricula_segura.py

1.2 Descrição da Implementação

Para a execução deste trabalho, utilizei a biblioteca `mysql-connector-python`. O processo seguiu as seguintes etapas:

- **Início da Transação:** Iniciei a unidade lógica através do comando `conexao.start_transaction(`
- **Consistência:** Realizei consultas prévias para validar a existência do aluno e do curso antes de qualquer inserção.
- **Atomicidade:** Implementei um tratamento de exceções que aciona o `conexao.rollback()` em caso de falha.
- **Durabilidade:** Finalizei o processo com o `conexao.commit()` para assegurar a persistência definitiva dos dados.

2 Evidências de Matrícula (Sucesso e Erro)

```
Arquivo    Editar    Exibir    gustavo@pop-os: ~/Documentos/bd2-lista1/aula-acid

    conexao = mysql.connector.connect(
        host="127.0.0.1",
        user="root",
        password="root123",
        database="aula1"
    )
    cursor = conexao.cursor()

    # 2. Início da Transação (START TRANSACTION)
    conexao.start_transaction()

    # Dados para o teste
    id_aluno_teste = 1
    id_curso_teste = 1
    disciplina_teste = "Sistemas Operacionais"
    semestre_teste = "2026.1"

    print(f"--- Iniciando matrícula do aluno {id_aluno_teste} ---")

    # Passo 1: Verificar se o aluno existe
    cursor.execute("SELECT COUNT(*) FROM ALUNOS WHERE id_aluno = %s", (id_aluno_teste,))
    if cursor.fetchone()[0] == 0:
        raise Exception("ERRO: Aluno não cadastrado.")

    # Passo 2: Verificar se o curso existe
    cursor.execute("SELECT COUNT(*) FROM CURSOS WHERE id_curso = %s", (id_curso_teste,))
    if cursor.fetchone()[0] == 0:
        raise Exception("ERRO: Curso não cadastrado.")

    # Passo 3: Inserir a matrícula
    sql = "INSERT INTO MATRICULAS (id_aluno, id_curso, semestre, disciplina) VALUES (%s, %s, %s, %s)"
    cursor.execute(sql, (id_aluno_teste, id_curso_teste, semestre_teste, disciplina_teste))

    # Passo 4: Confirmar com COMMIT
    conexao.commit()
    print("✅ Transação confirmada: Matrícula realizada com sucesso!")

except Exception as e:
    # Se ocorrer erro em QUALQUER etapa, executa o ROLLBACK
    if 'conexao' in locals() and conexao.is_connected():
        conexao.rollback()
        print(f"❌ Falha detectada: {e}")
EOF     conexao.close()als() and conexao.is_connected(): foi feita no banco. ---")
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ python3 matricula_segura.py
--- Iniciando matrícula do aluno 1 ---
✅ Transação confirmada: Matrícula realizada com sucesso!
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$
```

Figura 1: Confirmação do COMMIT no terminal após matrícula bem-sucedida.

```
Arquivo    Editar    Exibir        gustavo@pop-os: ~/Documentos/bd2-lista1/aula-acid

        database="aula1"
    )
    cursor = conexao.cursor()

    # 2. Início da Transação (START TRANSACTION)
    conexao.start_transaction()

    # Dados para o teste
    id_aluno_teste = 1
    id_curso_teste = 1
    disciplina_teste = "Sistemas Operacionais"
    semestre_teste = "2026.1"

    print(f"--- Iniciando matrícula do aluno {id_aluno_teste} ---")

    # Passo 1: Verificar se o aluno existe
    cursor.execute("SELECT COUNT(*) FROM ALUNOS WHERE id_aluno = %s", (id_aluno_teste,))
    if cursor.fetchone()[0] == 0:
        raise Exception("ERRO: Aluno não cadastrado.")

    # Passo 2: Verificar se o curso existe
    cursor.execute("SELECT COUNT(*) FROM CURSOS WHERE id_curso = %s", (id_curso_teste,))
    if cursor.fetchone()[0] == 0:
        raise Exception("ERRO: Curso não cadastrado.")

    # Passo 3: Inserir a matrícula
    sql = "INSERT INTO MATRICULAS (id_aluno, id_curso, semestre, disciplina) VALUES (%s, %s, %s, %s)"
    cursor.execute(sql, (id_aluno_teste, id_curso_teste, semestre_teste, disciplina_teste))

    # Passo 4: Confirmar com COMMIT
    conexao.commit()
    print("✅ Transação confirmada: Matrícula realizada com sucesso!")

except Exception as e:
    # Se ocorrer erro em QUALQUER etapa, executa o ROLLBACK
    if 'conexao' in locals() and conexao.is_connected():
        conexao.rollback()
        print(f"× Falha detectada: {e}")
EOF     conexao.close()als() and conexao.is_connected(): foi feita no banco. ---")
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ python3 matricula_segura.py
--- Iniciando matrícula do aluno 1 ---
✅ Transação confirmada: Matrícula realizada com sucesso!
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ python3 matricula_segura.py
--- Iniciando matrícula do aluno 999 ---
× Falha detectada: ERRO: Aluno não cadastrado.
--- ROLLBACK executado: Nenhuma alteração foi feita no banco. ---
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$
```

Figura 2: Execução do Rollback após tentativa de matricular aluno inexistente (999).

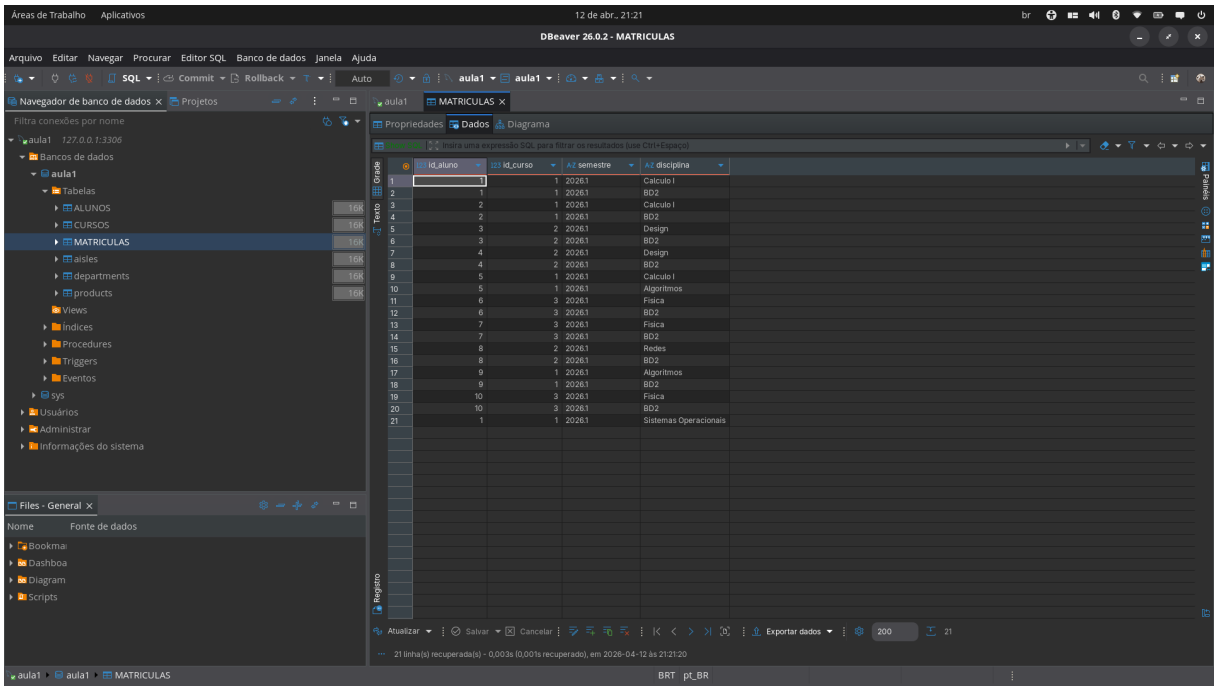

Figura 3: DBeaver comprovando a integridade dos dados durante os testes transacionais.

3 Cancelamento de Matrícula (Controle de Erro de Negócio)

Desenvolvi um segundo script para realizar a exclusão de registros, garantindo que a transação seja desfeita se a matrícula alvo não for encontrada.

```python
# Trecho principal da l gica de cancelamento
sql = "DELETE FROM MATRICULAS WHERE id_aluno = %s AND disciplina = %s"
cursor.execute(sql, (id_aluno_alvo, disciplina_alvo))

if cursor.rowcount == 0:
    raise Exception("Matricula nao encontrada para cancelamento.")
conexao.commit()
```

Listing 2: cancelamento.py

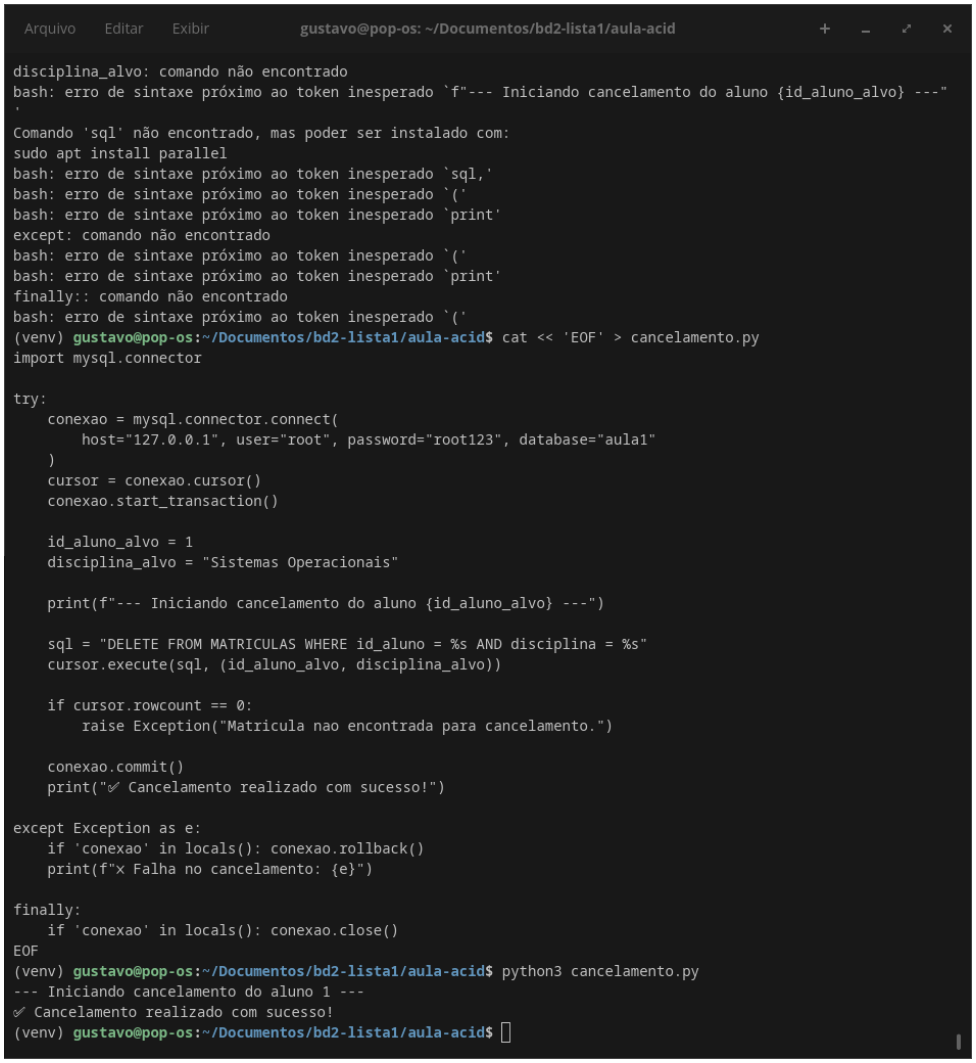

Figura 4: Cancelamento efetuado com sucesso (Primeira execução).

```
Arquivo   Editar   Exibir        gustavo@pop-os: ~/Documentos/bd2-lista1/aula-acid

Comando 'sql' não encontrado, mas poder ser instalado com:
sudo apt install parallel
bash: erro de sintaxe próximo ao token inesperado `sql,'
bash: erro de sintaxe próximo ao token inesperado `('
bash: erro de sintaxe próximo ao token inesperado `print'
except: comando não encontrado
bash: erro de sintaxe próximo ao token inesperado `('
bash: erro de sintaxe próximo ao token inesperado `print'
finally:: comando não encontrado
bash: erro de sintaxe próximo ao token inesperado `('
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ cat << 'EOF' > cancelamento.py
import mysql.connector

try:
    conexao = mysql.connector.connect(
        host="127.0.0.1", user="root", password="root123", database="aula1"
    )
    cursor = conexao.cursor()
    conexao.start_transaction()

    id_aluno_alvo = 1
    disciplina_alvo = "Sistemas Operacionais"

    print(f"--- Iniciando cancelamento do aluno {id_aluno_alvo} ---")

    sql = "DELETE FROM MATRICULAS WHERE id_aluno = %s AND disciplina = %s"
    cursor.execute(sql, (id_aluno_alvo, disciplina_alvo))

    if cursor.rowcount == 0:
        raise Exception("Matricula nao encontrada para cancelamento.")

    conexao.commit()
    print("✅ Cancelamento realizado com sucesso!")

except Exception as e:
    if 'conexao' in locals(): conexao.rollback()
    print(f"❌ Falha no cancelamento: {e}")

finally:
    if 'conexao' in locals(): conexao.close()
EOF
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ python3 cancelamento.py
--- Iniciando cancelamento do aluno 1 ---
✅ Cancelamento realizado com sucesso!
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$ python3 cancelamento.py
--- Iniciando cancelamento do aluno 1 ---
❌ Falha no cancelamento: Matricula nao encontrada para cancelamento.
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/aula-acid$
```

Figura 5: Falha e Rollback ao tentar cancelar matrícula já excluída (Segunda execução).

4 Consultas Acadêmicas (DQL)

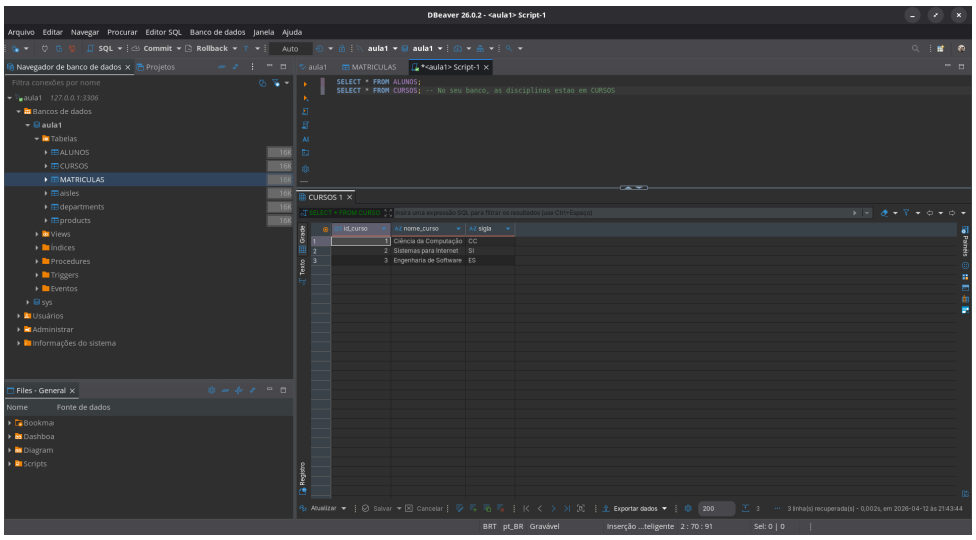

Figura 6: Listagem geral de ALUNOS e CURSOS.

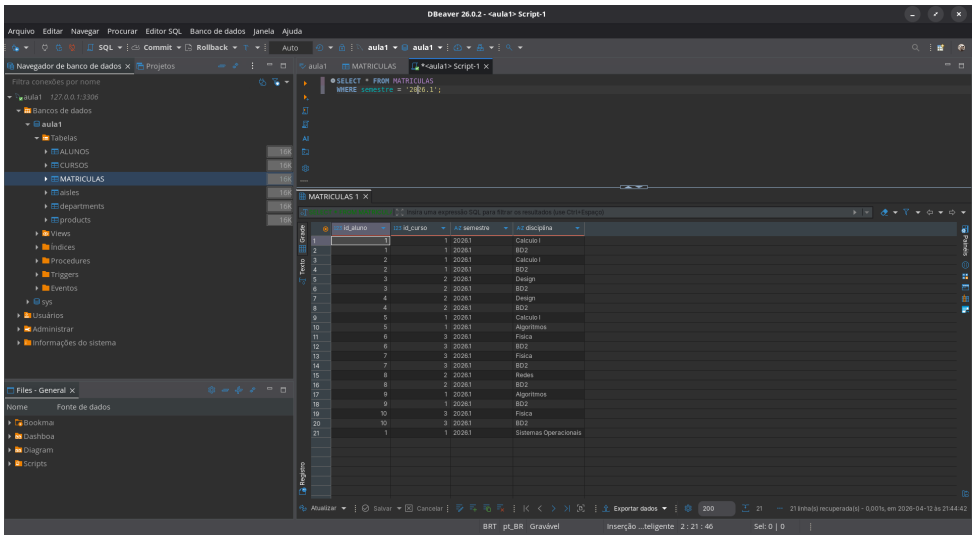

Figura 7: Matrículas filtradas pelo semestre 2026.1.

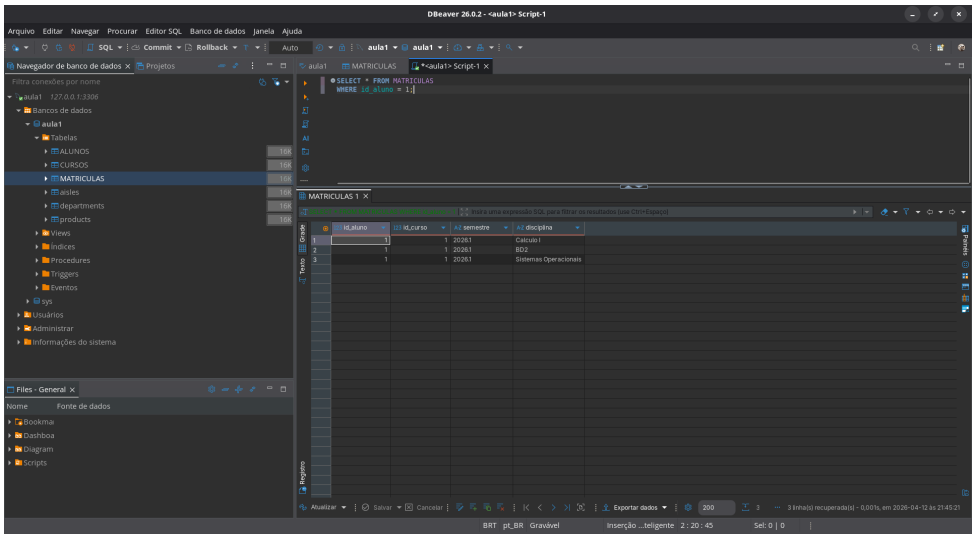

Figura 8: Consulta específica por ID de aluno.

5 Respostas Teóricas (Slide 44)

1. **Por que a atomicidade é essencial em transferências bancárias?**
   Ela garante que o débito em uma conta e o crédito na outra ocorram como uma única operação. Se o sistema falhar no meio do caminho, o dinheiro não se perde nem é duplicado.

2. **Como a consistência se relaciona com a integridade referencial?**
   A consistência assegura que o banco respeite todas as regras predefinidas (como chaves estrangeiras). Uma transação não pode deixar uma matrícula vinculada a um aluno que não existe.

3. **Por que o isolamento é importante em sistemas multiusuário?**
   Para evitar que transações simultâneas interfiram entre si, prevenindo problemas como leituras sujas ou atualizações perdidas.

4. **O que pode acontecer se não houver durabilidade?**
   Dados confirmados via `COMMIT` poderiam ser perdidos em caso de queda de energia ou reinicialização do servidor.

5. **Exemplo prático de transação:**
   O processo de matrícula realizado neste relatório: validação de dados, inserção e confirmação (ou cancelamento em caso de erro).

6 Conclusão

Esta prática demonstrou que as propriedades ACID são fundamentais para a segurança de sistemas transacionais. A implementação correta de transações garantiu que o banco `aula1` não aceitasse dados inválidos ou incompletos.